Stock Portfolio Risk & Forecasting Platform

Business Requirements Document

A Django REST Framework backend with JWT-secured, role-based portfolio management, a machine-learning risk and forecast engine, live risk monitoring over a WebSocket, admin web pages for data and model training, and a Flutter client used to test the platform on phones, emulators and the browser.

Prepared by: Dion James Smith
Version 3.0 — November 2026
Supersedes Version 2.0 (November 2026)

Table of Contents

Document Changes from Version 2.0 4
1. Executive Summary 4
2. Project Objectives 5
3. Technology Stack 5
4. System Architecture Overview 6
4.1 Layers . . . 6
4.2 Clients . . . 6
4.3 Request flow . . . 6
4.4 Background flow . . . 6
4.5 Live event flow . . . 6
5. Application (Django App) Structure 7
6. Database Design 7
6.1 Entity overview . . . 7
6.2 users . . . 8
6.3 portfolios . . . 8
6.4 holdings . . . 8
6.5 price_history . . . 9
6.6 dataset_uploads . . . 9
6.7 training_runs . . . 9
6.8 Relationships . . . 9
7. Authentication and Account Creation 10
7.1 Custom user model . . . 10
7.2 How each role is created . . . 10
7.3 Token lifecycle . . . 10
7.4 Token lifetimes and limits . . . 11
7.5 Authorization model . . . 11
8. Roles and Permission Matrix 11
9. API Endpoint Reference 12
9.1 Accounts — /api/accounts/ . . . 12
9.2 Portfolios and holdings — /api/ . . . 12
9.3 Market data — /api/market/ . . . 12
9.4 Analytics — /api/analytics/ . . . 13
9.5 Dashboards — /api/dashboard/ . . . 13
9.6 Live monitoring — /ws/live/ . . . 13
10. API Response Contract 13
11. Django Web Interface 14
11.1 Dataset upload requirements . . . 14
11.2 Model training requirements . . . 15
12. Live Risk Monitoring (WebSocket API) 16
12.1 Purpose and framing . . . 16
12.2 Connection and authentication . . . 16
12.3 Messages from the client . . . 16
12.4 Events from the server . . . 17
12.5 Event format . . . 17
12.6 Delivery and scoping . . . 17
12.7 Live price source . . . 18
12.8 Session and connection rules . . . 18
12.9 Backend requirements . . . 18
12.10 Acceptance criteria . . . 19
13. Flutter Test Client Requirements 19
13.1 Purpose and scope . . . 19
13.2 Test targets and base URLs . . . 19
13.3 Backend requirements for Flutter . . . 20
13.4 App architecture . . . 20
13.5 Authentication and session requirements . . . 21
13.6 Screen inventory — common . . . 21
13.7 Screen inventory — investor . . . 22
13.8 Screen inventory — analyst . . . 22

13.9 Screen inventory — admin . . . 23
13.10 Page design requirements . . . 23
13.11 Acceptance criteria for the Flutter test . . . 23
14. Celery and Redis Task Design 24
14.1 Redis databases . . . 24
14.2 Scheduled and on-demand tasks . . . 24
14.3 Caching . . . 25
15. Machine Learning Pipeline 25
15.1 Code location . . . 25
15.2 Features . . . 25
15.3 Model 1 — Risk classification . . . 25
15.4 Model 2 — Direction forecast . . . 25
15.5 Training method and evaluation . . . 26
15.6 Versioning . . . 26
16. Dataset Source and Ingestion 26
17. Environment Configuration 27
18. Project Directory Structure 27
19. Security and Non-Functional Requirements 28
20. Delivery Roadmap and Future Scope 28
Appendix A — Sample Flutter Test Flow 29
Appendix B — Machine Learning Disclaimer 29

Document Changes from Version 2.0

Area	Version 2.0	Version 3.0
Positioning	Portfolio risk and forecasting platform	Live risk monitoring: daily ML predictions plus live updates pushed over a WebSocket (Section 12)
Phase status	Phases 0–9B complete; Phase 10 next	Phases 0–10 complete; Phase 10B (WebSocket API) next; Phase 11 extended with live screens
Apps	Seven apps	Eight apps: adds monitoring (WebSocket consumer, live events, live price task)
Server	WSGI (runserver; Gunicorn planned for containers)	ASGI with Daphne: runserver serves HTTP and WebSocket; Daphne in containers
Redis	Four databases	Five databases: adds the channel layer (database 4)
Flutter	REST only; recompute progress by polling	REST plus one WebSocket; prediction.updated replaces polling (polling kept as a fallback)
Live prices	Future scope (Phase 13)	Phase 10B: every 2 minutes in US market hours, cached in Redis, never written to price_history
Risk alerts	Push notifications in future scope	risk.changed over the WebSocket while the app is open; push notifications remain future scope for a closed app
Views rule	Function-based views only	Function-based REST and web views; one small class-based WebSocket consumer (required by Django Channels)

1. Executive Summary

This platform lets investors track stock portfolios, view stored market data, and receive machine-learning risk classifications and next-day price direction forecasts for each holding. It is built with Django and Django REST Framework, secured with a custom JWT scheme on a purpose-built User model, and organised around three roles — investor, analyst and admin — each with distinct data access rules.

Two supervised models sit at the core of the analytics layer: a Random Forest classifier that labels each holding's risk as low, medium or high, and a Gradient Boosting classifier that predicts whether the next trading day's close will be up or down. Each model is compared against simple baselines so its value can be judged honestly.

The platform is framed as live risk monitoring. The models do the slow thinking: they work on daily prices and refresh every holding's prediction each night, after training, or when an analyst asks. A JWT-authenticated WebSocket delivers what changes in between — live prices for the tickers a user follows, an alert the moment a holding's risk label changes, and a notice when a requested recompute finishes. The WebSocket does not stream predictions: a price move during the day does not change tomorrow's prediction until the next daily run.

Admins control the data and the models from Django web pages: they upload the historical price dataset, start training runs, follow progress, and review every saved model version. Celery and Redis handle all slow work in the background — dataset imports, daily and live price updates, nightly predictions, weekly retraining — and Redis also caches expensive summaries, enforces rate limits and carries WebSocket events. PostgreSQL is the system of record, accessed only through the Django ORM.

The project will be tested through a Flutter app. Every investor, analyst and admin feature that Flutter needs is available through the JSON API and the WebSocket. Section 12 specifies the WebSocket

API and Section 13 specifies the Flutter screens, how they map to the API, how they must be designed for phones first, and the backend settings needed for an emulator, a physical phone and Chrome to reach the server.

2. Project Objectives

- **Custom authentication** — a dedicated accounts app with its own User model, JWT access and refresh tokens built with PyJWT, logout blacklisting and role-based, object-level authorization.
- **Relational database design** — normalised tables with clear foreign keys, including a self-referencing analyst assignment, and ORM joins and aggregations instead of duplicated fields.
- **Explainable machine learning** — two well-understood algorithms with leak-free features, time-based validation and baseline comparison, integrated into live APIs and background tasks.
- **Live risk monitoring** — a WebSocket that pushes live prices, risk-change alerts and recompute results, authenticated with the same JWT and scoped by role, built with Django Channels and a Redis channel layer.
- **Admin control through the GUI** — dataset upload and model training are done from Django templates, not the command line.
- **Asynchronous design** — Celery and Redis for scheduled ingestion, live prices, predictions, retraining, uploads, caching, throttling and event delivery.
- **API-first design for Flutter** — consistent JSON responses and events so a Flutter app can be built and tested against the backend without backend changes.

3. Technology Stack

Layer	Technology	Purpose
Web / API framework	Django 6.1 + Django REST Framework 3.18	Function-based API views, routing, serializers for output
Real-time	Django Channels 4.3 + channels-redis 4.3	WebSocket consumer and the channel layer that carries events from workers to connections
ASGI server	Daphne 4.2	Serves HTTP and WebSocket; used by runserver and in containers
Web interface	Django templates, Bootstrap 5, Chart.js	Session-based pages, admin dataset and training pages
Database	PostgreSQL (psycopg 3)	Users, portfolios, holdings, price history, uploads, training runs
Data access	Django ORM	All reads and writes; no raw SQL
Authentication	Custom JWT with PyJWT	Access and refresh tokens, each signed with its own secret; same access token for the WebSocket
Task queue	Celery 5.6 + Celery Beat	Scheduled and on-demand background work
Broker / cache	Redis	Broker, results, cache, throttles, token blacklist and channel layer (separate databases)
Machine learning	scikit-learn, pandas, NumPy, joblib	Random Forest, Logistic Regression baseline, Gradient Boosting
Market data	Kaggle CSV dataset, yfinance	Ten-year history, daily updates and live quotes
Test client	Flutter (Android, iOS, web)	Primary client used to test the platform (Section 13)

4. System Architecture Overview

4.1 Layers

- **Client layer** — the Flutter app (JWT over REST and WebSocket), the Django web pages (session login) and the Postman collection.
- **API and web layer** — function-based DRF API views and Django template views. JWT authentication and permission checks run before any business logic.
- **Real-time layer** — Daphne (ASGI) and one Channels consumer at /ws/live/. A Redis channel layer carries events from Celery workers to open connections.
- **Business logic layer** — forms and ModelForms validate input (WebSocket messages included); services and selectors hold the logic; serializers shape output.
- **Data layer** — PostgreSQL through the ORM, with select_related, prefetch_related and annotate to keep queries efficient.
- **Asynchronous layer** — Celery workers and Celery Beat with Redis, for imports, price updates, live prices, predictions, training and cache invalidation.

4.2 Clients

Client	Who uses it	How it connects	What it covers
Flutter app	Investors, analysts, admins	JSON API with Bearer JWT; WebSocket /ws/live/?token=	Everything except file upload and starting training; live prices and alerts
Django web pages	All roles; admin pages for admins	Browser session login	Dashboards, portfolios, users, Datasets, ML Training, Model versions
Postman collection	Developer	JSON API with Bearer JWT	Automated end-to-end API check; prepares WebSocket URLs and triggers live events

4.3 Request flow

Client → JWT authentication (or session) → role and object-level checks → view → form validation → service / ORM → PostgreSQL → serializer → standard JSON response. Summary and dashboard endpoints read Redis first and fall back to the database.

4.4 Background flow

Celery Beat and user actions put jobs on the Redis queue. Workers import uploaded files, fetch prices, compute predictions and train models, write results through the ORM, save models to the registry folder, and clear the related cache entries so the next request shows fresh data. When a job changes something a user is watching, the worker also publishes a live event (Section 4.5).

4.5 Live event flow

Celery task (live price poll, price download or prediction run) → event built in monitoring.services → Redis channel layer group (user_<id>, admins or ticker_<TICKER>) → each connected consumer checks that its token is still valid → JSON event to the app → the app updates the screen. REST stays the source of truth: the app loads screens through the API and applies events on top.

5. Application (Django App) Structure

App	Owns	Responsibility
accounts	User	Registration, login, JWT tokens, logout, profile, password change, role and analyst assignment, account web pages
core	—	Shared code: logger, decorators (@log_execution_time, @admin_required), api_response, permissions, pagination, exception handler, throttles, cache keys, multiple-file field, ping task
market_data	PriceHistory, DatasetUpload	Dataset upload pages and import task, Yahoo Finance daily updates and live quotes, latest price, history and refresh APIs
portfolios	Portfolio, Holding	Role-scoped CRUD APIs, portfolio summary
ml_training	TrainingRun	Algorithms, features, labels, data split, training, evaluation, model registry, training and model version pages
analytics	—	Loads the newest models, writes predictions to holdings, risk and forecast APIs, portfolio risk summary, task status
dashboards	—	Investor, analyst and admin dashboards (API and web), holding trend, portfolio and holding pages
monitoring	— (no tables)	WebSocket consumer and routing, live event publishing, live price task

REST and web views are function-based. The one WebSocket consumer is a small class, because Django Channels only supports class-based consumers; it has connect, disconnect, receive and one event handler.

6. Database Design

6.1 Entity overview

Table	App	Represents
users	accounts	Every person: investor, analyst or admin
portfolios	portfolios	A named collection of holdings owned by one user
holdings	portfolios	One stock position plus its latest ML outputs
price_history	market_data	Daily OHLCV prices per ticker, shared by every holding of that ticker
dataset_uploads	market_data	One dataset upload: status, progress and per-file results
training_runs	ml_training	One training run: options, status, current step, new model versions and results

Phase 10B adds no tables. Live quotes and WebSocket messages live in Redis (Section 14.1).

6.2 users

Field	Type	Notes
id	Integer	Primary key
username, email	String	Unique (case-insensitive checks)
password	Hashed string	Never stored or returned in plain text
role	Choice	investor / analyst / admin; admin also sets is_staff and is_superuser
assigned_analyst_id	FK to users.id	Nullable; only investors keep one; must be an active analyst
is_active, date_joined, updated_at	Boolean / DateTime	Standard account fields

6.3 portfolios

Field	Type	Notes
id	Integer	Primary key
user_id	FK to users.id	Owner
name	String	Required; unique per owner
description	Text	Optional
created_at, updated_at	DateTime	

6.4 holdings

Field	Type	Notes
id, portfolio_id	Integer / FK	Belongs to one portfolio; deleted with it
ticker	String	Stored uppercase; cannot be changed after creation; joined to price_history
quantity, buy_price	Decimal	Must be greater than zero
buy_date	Date	Cannot be in the future
risk_label, risk_score	Choice / Float	low / medium / high and its probability; written only by tasks
predicted_direction, prediction_confidence	Choice / Float	up / down and its probability; written only by tasks
last_predicted_at	DateTime	Time of the latest prediction
risk_model_version, forecast_model_version	String	Model versions that produced the outputs
created_at, updated_at	DateTime	

A change of risk_label between two prediction runs is what triggers a risk.changed event (Section 12.4).

6.5 price_history

Field	Type	Notes
ticker, date	String / Date	Unique together
open_price, high_price, low_price, close_price	Decimal (12,4)	Validated: positive, high ≥ low
volume	BigInteger	Nullable when the source has no volume
created_at	DateTime	

price_history has no foreign key to holdings: prices belong to a ticker and are shared by every investor holding it. They are joined at query time on the ticker. Live intraday prices are never written here; the table only holds finished trading days, so the ML features stay leak-free.

6.6 dataset_uploads

Field	Type	Notes
uploaded_by_id	FK to users.id	Admin who uploaded; kept as null if the user is deleted
name, tickers	String	Summary of the files; optional ticker filter
status	Choice	pending / processing / completed / failed
total_files, files_processed	Integer	Drive the progress bar
tickers_loaded, rows_inserted, rows_skipped	Integer	Result counts
results	JSON	One entry per file: ticker, status, rows added and skipped, message
error, task_id, created_at, started_at, finished_at	Text / String / DateTime	

6.7 training_runs

Field	Type	Notes
model_choice	Choice	all / risk / forecast
tickers, update_predictions	String / Boolean	Optional ticker filter; whether to recompute holdings afterwards
triggered_by_id	FK to users.id	Null for the weekly scheduled run
status, current_step	Choice / String	pending / running / completed / failed and the step in progress
risk_version, forecast_version	String	Versions saved by this run
result, error	JSON / Text	Metric summaries, prediction counts, error message
task_id, created_at, started_at, finished_at	String / DateTime	

6.8 Relationships

- users (1) — (many) portfolios (1) — (many) holdings — matched by ticker — price_history.

- users (1) — (many) users through assigned_analyst_id (analyst to investors). The same link decides which analyst receives a holding's live events.
- users (1) — (many) dataset_uploads and training_runs.

7. Authentication and Account Creation

7.1 Custom user model

The project defines its own User model from the first migration, with role and assigned_analyst as real fields, and sets it as AUTH_USER_MODEL.

7.2 How each role is created

Public registration always creates an investor. Nobody chooses their own role; a role field in the request is ignored. This prevents anyone from signing up as an analyst or admin.

Role	How it is created
Investor	POST /api/accounts/register/ or the Register page
Analyst	Register normally, then an admin sets role to analyst (PATCH /api/accounts/users/{id}/ or the Users page)
Admin	python manage.py createsuperuser, or an admin sets role to admin

Registration request body (same for every new user):

```
POST /api/accounts/register/
{
    "username": "dion_investor",
    "email": "dion.investor@example.com",
    "password": "Investor-Pass-2026!",
    "password_confirmation": "Investor-Pass-2026!"
}
```

Promotion and assignment by an admin (Authorization: Bearer <admin access token>):

```
PATCH /api/accounts/users/{analyst_id}/      {"role": "analyst"}
PATCH /api/accounts/users/{investor_id}/     {"assigned_analyst": <analyst_id>}
```

- An admin cannot change their own role.
- Changing an analyst to another role unassigns all of their investors.
- A user who stops being an investor loses their assigned analyst.

7.3 Token lifecycle

- Login with username or email returns the user (including role) and an access/refresh token pair.
- Requests send Authorization: Bearer <access token>.
- Refresh exchanges a valid refresh token for a new access token.
- Logout blacklists the refresh token in a separate Redis database until it would have expired.
- Password change invalidates every older token and returns a new pair.
- Access and refresh tokens use different secrets and carry a token_type claim, so one cannot be used as the other.
- The WebSocket uses the same access token in its URL. When the token expires the socket closes with code 4401; the app refreshes the token and reconnects.

7.4 Token lifetimes and limits

Item	Value
Access token	15 minutes
Refresh token	7 days
Login rate limit	5 per minute per IP
Registration rate limit	10 per hour per IP
Price refresh rate limit	10 per hour per user
Tickers followed per WebSocket connection	20

7.5 Authorization model

- **Role checks** — some endpoints are limited to a role (for example only admins manage users).
- **Object-level checks** — querysets are filtered so investors see only their own records and analysts only their assigned investors' records. A record outside the user's scope returns 404, which prevents guessing other users' IDs. A visible record the user may not change returns 403.
- **Live events** — holding events are sent only to the owner, the owner's assigned analyst and admins, following the same rules as the API.
- **Admin web pages** — the @admin_required decorator protects the Datasets, ML Training and Users pages and sends other roles back to their dashboard.

8. Roles and Permission Matrix

Action	Investor	Analyst	Admin
Register, log in, edit own profile	Yes	Yes	Yes
View own portfolios and holdings	Yes	—	Yes
View assigned investors' portfolios	No	Yes (read-only)	Yes
View all portfolios	No	No	Yes
Create / edit / delete portfolios and holdings	Own only	No	Yes
View prices and price history	Yes	Yes	Yes
Queue a price refresh	No	Yes	Yes
View risk, forecast, risk summary, trend	Own	Assigned	All
Queue a prediction recompute, check task status	No	Yes	Yes
Open the live connection and follow ticker prices	Yes	Yes	Yes
Receive risk.changed and prediction.updated	Own holdings	Assigned investors' holdings	All holdings
Investor / analyst / admin dashboard	Investor	Analyst	Admin
List users, change roles, assign analysts	No	No	Yes
Upload datasets, train models, view model versions (web)	No	No	Yes

9. API Endpoint Reference

All REST endpoints are under /api/ and every response uses the format in Section 10. The WebSocket endpoint is /ws/live/ and is specified in Section 12.

9.1 Accounts — /api/accounts/

Method	Endpoint	Access	Purpose
POST	register/	Public	Create an investor
POST	login/	Public	username_or_email + password → user + tokens
POST	token/refresh/	Refresh token	New access token
POST	logout/	Authenticated	Blacklist the refresh token
GET / PATCH	me/	Authenticated	Own profile; PATCH username / email
POST	change-password/	Authenticated	New password and new token pair
GET	users/	Admin	Paginated user list
PATCH	users/{id}/	Admin	Change role or assigned analyst

9.2 Portfolios and holdings — /api/

Method	Endpoint	Access	Purpose
GET / POST	portfolios/	All (scoped) / investor, admin	List visible portfolios; create (admin may pass user_id)
GET / PATCH / DELETE	portfolios/{id}/	Visible / owner, admin	Detail, rename, delete
GET	portfolios/{id}/summary/	Visible	Cost, value, gain/loss, risk distribution, per-holding rows
GET / POST	portfolios/{id}/holdings/	Visible / owner, admin	List or add holdings
GET / PATCH / DELETE	holdings/{id}/	Visible / owner, admin	Holding detail, edit, delete

9.3 Market data — /api/market/

Method	Endpoint	Access	Purpose
GET	prices/{ticker}/	Authenticated	Latest stored daily price
GET	prices/{ticker}/history/?start=&end=	Authenticated	Paginated history, newest first
POST	refresh/{ticker}/	Analyst, admin	Queue a price download (202 + task_id); new rows also send price.update

9.4 Analytics — /api/analytics/

Method	Endpoint	Access	Purpose
GET	holdings/{id}/risk/	Visible	Risk label, score, model version, holding age, gain/loss %
GET	holdings/{id}/forecast/	Visible	Direction, confidence, model version
POST	holdings/{id}/recompute/	Analyst, admin	Queue a prediction (202 + task_id); the result also arrives as prediction.updated
GET	portfolio/{id}/risk-summary/	Visible	Risk and direction distribution, average confidence, total value
GET	tasks/{task_id}/	Analyst, admin	Celery state and result (fallback when no live event arrives)

9.5 Dashboards — /api/dashboard/

Method	Endpoint	Access	Purpose
GET	investor/	Investor	Portfolios, totals, risk distribution
GET	analyst/	Analyst	Clients, totals, high-risk holdings
GET	admin/	Admin	User counts, totals, latest price date, models in use
GET	holdings/{id}/trend/?start=&end=	Visible	Price points for a chart (default last 180 days)

9.6 Live monitoring — /ws/live/

Protocol	Endpoint	Access	Purpose
WebSocket	ws/live/?token=<access token>	Authenticated	Live prices, risk-change alerts and recompute results (Section 12)

10. API Response Contract

Every API response — success or error — has the same four keys. Flutter reads the same shape everywhere. WebSocket events use the same four keys plus type (Section 12.5).

```
{
    "status": "success" | "error",
    "message": "Human-readable message",
    "data": { ... } | [ ... ] | null,
    "errors": null | { "field_name": ["Error message"] }
}
```

Topic	Rule
Pagination	data contains count, next, previous and results; 20 per page, ?page_size= up to 100; next and previous use the caller's host
Decimals	Prices, quantities and money values are strings (for example "150.0000") to avoid rounding errors
Dates and times	Dates are YYYY-MM-DD; date-times are ISO 8601 in UTC
Cache flag	Summary, risk-summary and dashboard responses include from_cache

Topic	Rule
Background jobs	202 Accepted with data.task_id; the same task_id is carried by the prediction.updated event
Trailing slash	Every endpoint ends with /

Status code	Meaning	What the client should do
200 / 201	Success / created	Show the data
202	Job queued	Show 'in progress' and wait for the live event (or poll the task status)
400	Validation failed	Show errors under each form field
401	Token missing, invalid or expired	Refresh once and retry; if that fails, go to login
403	Role not allowed	Hide the action or show a 'not permitted' message
404	Not found or not visible to this user	Show a not-found message
429	Rate limit	Wait errors.retry_after_seconds before retrying
500 / 503	Server error / task queue unavailable	Show a retry option

11. Django Web Interface

The web interface uses session login and the same role rules as the API. The admin pages below replace the command-line tools of version 1.0: an admin controls data and training entirely from the browser.

URL	Who	Page
/, /register/, /login/	Everyone	Home and account pages
/dashboard/	All roles	Role-specific dashboard
/profile/, /profile/password/	All roles	Profile and password change
/portfolios/, /portfolios/{id}/, /holdings/{id}/	All roles (scoped)	Portfolios, holdings, price chart
/users/	Admin	Role changes and analyst assignment
/datasets/, /datasets/{id}/	Admin	Upload CSV files, a folder or a zip; progress; per-file results; fetch latest prices
/ml/, /ml/runs/{id}/	Admin	Data readiness, start training, live run progress, cancel, history
/ml/models/, /ml/models/{type}/{version}/	Admin	Every model version with metrics, baselines, confusion matrix, feature importance
/admin/	Admin	Django admin

11.1 Dataset upload requirements

- Accept several CSV files, a whole folder, or the Kaggle zip as downloaded; ignore files that are not .csv or .zip.
- Each file is named after its ticker and has Date, Open, High, Low, Close and optionally Volume columns.

- Optional ticker filter.
- Import runs in Celery; the page shows files processed / total and refreshes itself.
- Already-stored rows are skipped, so repeat uploads are safe; uploaded files are deleted after import.
- Each file's result is shown: rows added, rows skipped, or the reason it failed.

11.2 Model training requirements

- Show whether stored prices cover the training, validation and test periods; training is disabled until they do.
- Train both models, the risk model only, or the forecast model only, optionally for selected tickers.
- Optionally update every holding's prediction when training finishes (labels that move send risk.changed).
- Only one active run at a time; a run waiting for a worker can be cancelled.
- Show the current step while running and the test metrics against baselines when finished.
- Keep every model version; the newest of each type is used for predictions.

12. Live Risk Monitoring (WebSocket API)

This section specifies the WebSocket the Flutter app uses to stay up to date without refreshing. It is delivered in Phase 10B by the monitoring app and changes no REST endpoint.

12.1 Purpose and framing

Part	How often	What it produces
ML models	Daily: nightly runs (23:00 and 23:15 UTC), after training, or on an analyst's recompute	A risk label and a next-day direction for every holding
WebSocket	As it happens	Live prices for followed tickers, risk-change alerts, finished recomputes

- The WebSocket does not stream predictions. A price change during the day does not change a prediction until the next daily run.
- Live prices are near-live: every 2 minutes during US market hours from Yahoo Finance, which is unofficial and rate-limited.
- REST stays the source of truth. Screens load through the API; events update them afterwards.

12.2 Connection and authentication

Target	WebSocket URL
Android emulator	ws://10.0.2.2:8000/ws/live/?token=<access token>
iOS simulator, Flutter web	ws://127.0.0.1:8000/ws/live/?token=<access token>
Physical phone (same Wi-Fi)	ws://<computer IP>:8000/ws/live/?token=<access token>
Deployment (Phase 12)	wss://<domain>/ws/live/?token=<access token>

- The access token travels in the URL because browsers cannot set an Authorization header on a WebSocket. Daphne logs only the path, so the token is never written to the server log.
- The token passes the same checks as REST: valid signature, token_type access, active user, issued after the last password change.
- If any check fails, the server sends an error event and closes the connection with code 4401.
- There is no Origin check. Flutter on Android and iOS sends no Origin header, and Channels' origin validator rejects such connections. Because the socket is authenticated by a token rather than a cookie, another website cannot use a user's session, which is the attack an Origin check prevents.

12.3 Messages from the client

action	Example	Rules
subscribe	{"action": "subscribe", "tickers": ["AAPL", "MSFT"]}	Same ticker pattern as the REST API; upper-cased; at most 20 tickers per connection
unsubscribe	{"action": "unsubscribe", "tickers": ["MSFT"]}	Stops price.update for those tickers
ping	{"action": "ping"}	Every 30 seconds; the server re-checks the account and answers pong

Messages are validated by a Django form. Invalid JSON, an unknown action or an invalid ticker returns an error event with field errors, and the connection stays open.

12.4 Events from the server

type	Sent when	data
connection.ready	Right after a successful connection	user_id, username, role, token_expires_at, max_tickers
subscribed / unsubscribed	After subscribe or unsubscribe	tickers now followed
price.update	On subscribe (latest known price); every live poll; after new daily rows are saved	ticker, price, previous_close, change, change_percent, date, as_of, source (live / daily)
risk.changed	A prediction run changed a holding's risk_label	holding_id, portfolio_id, portfolio_name, owner, ticker, previous_label, risk_label, risk_score, model versions, direction, confidence, last_predicted_at
prediction.updated	A recompute task finished	The holding fields above plus the task_id returned by the recompute API
pong	Reply to ping	server_time, token_expires_at
error	Invalid message, expired token, or changed account	errors explains the problem

12.5 Event format

Every event uses the REST response shape plus type. Prices are strings, as in the REST API.

```
{
    "type": "price.update",
    "status": "success",
    "message": "New price for AAPL.",
    "data": {
        "ticker": "AAPL",
        "price": "231.4500",
        "previous_close": "228.1000",
        "change": "3.3500",
        "change_percent": "1.47",
        "date": "2026-11-09",
        "as_of": "2026-11-09T15:32:00.118204Z",
        "source": "live"
    },
    "errors": null
}
```

12.6 Delivery and scoping

Group	Who joins	Events
user_<id>	Every connection, for its own user	Holding events for the owner and for the assigned analyst
admins	Admin connections	Holding events for every holding
ticker_<TICKER>	Connections that subscribed to the ticker	price.update

- Holding events go to the owner, the owner's assigned analyst and the admins group, and to nobody else.
- Recipients are worked out when the event is sent, so assigning or removing an analyst takes effect immediately.
- risk.changed is sent only when the label actually changes; an unchanged label sends nothing.

12.7 Live price source

- monitoring.tasks.poll_live_prices runs every 2 minutes, Monday to Friday, 13:00–21:58 UTC, which covers US market hours in summer and winter time.
- It reads the latest price and previous close of every ticker held in a portfolio (yfinance fast_info).
- Each quote is kept in the Redis cache for 24 hours (key live_quote:<ticker>) so a new subscriber gets the latest price at once.
- Quotes are never written to price_history; the 22:00 UTC daily download still stores the final close.
- fetch_daily_prices and refresh_single_ticker send a daily price.update whenever they save new rows.

12.8 Session and connection rules

Situation	Server	Client
Access token expires	Next message or event closes with 4401	Refresh the token once and reconnect; log out if refresh fails
Password changed, account deactivated or role changed	Next ping closes with 4401	Same as above
Network drop or server restart	—	Reconnect after 1, 2, 4, 8, 16, then every 30 seconds; subscribe again
App goes to the background	Connection may be closed by the OS	Reconnect when the app returns; reload the visible screen through REST

12.9 Backend requirements

ID	Requirement
WS-01	Daphne is the first entry in INSTALLED_APPS and ASGI_APPLICATION points to stock_risk_engine.asgi, so runserver serves HTTP and WebSocket.
WS-02	asgi.py routes http to Django and websocket to monitoring.routing (/ws/live/).
WS-03	The channel layer uses its own Redis database (CHANNEL_LAYER_REDIS_URL), included in the check that all Redis URLs differ.
WS-04	The channel layer's Redis connection uses socket_timeout 10. redis-py 8 times out reads after 5 seconds by default while the channel layer waits up to 5 seconds, so idle connections would otherwise fail with error 1011.
WS-05	The consumer authenticates the token with the same functions as the REST API and closes with 4401 on failure.
WS-06	Client messages are validated by a Django form; errors use the field-list format of Section 10.
WS-07	Events use type plus status / message / data / errors; decimals are strings and date-times ISO 8601 UTC.
WS-08	Holding events are scoped to owner, assigned analyst and admins (Section 12.6).
WS-09	Prediction runs send risk.changed only for labels that changed; recompute_single_holding sends prediction.updated with its task_id.
WS-10	The live price task runs on the Beat schedule in Section 14.2 and caches quotes in Redis only.
WS-11	Token expiry is checked on every message and event; account changes are checked on every ping.
WS-12	No REST endpoint changes; the Postman collection keeps passing.

12.10 Acceptance criteria

- Missing, invalid, expired and refresh tokens are rejected with an error event and close code 4401.
- Investor, analyst and admin receive connection.ready with their role; the emulator host without an Origin header and Flutter web from localhost:3000 both connect.
- Invalid JSON, unknown actions, invalid tickers and more than 20 tickers return field errors without closing the connection.
- Subscribing returns the latest price at once; users who did not subscribe receive no price.update.
- An analyst recompute sends prediction.updated with the REST task_id to the investor, the analyst and admins, and nothing to other investors.
- A forced label change sends risk.changed with previous_label; a second run with the same label sends no alert.
- Password change and role change close the socket on the next ping; an idle connection stays open beyond 25 seconds.

13. Flutter Test Client Requirements

The platform will be tested by running a Flutter app against the API and the WebSocket. The Flutter pages must therefore be designed for the backend exactly as it is, for phone screens first, and for each role separately. This section is the specification for that app and for the backend settings it needs.

13.1 Purpose and scope

- Exercise every investor and analyst feature, and the admin dashboard and user management, through the JSON API.
- Show live prices, risk-change alerts and recompute results through the WebSocket (Section 12).
- Run on the Android emulator, a physical phone on the same Wi-Fi, the iOS simulator (on macOS) and Chrome.
- Built as a separate Flutter project; it never connects to the database directly.
- Dataset upload and starting training are done in the Django web pages; the Flutter admin links to them.

13.2 Test targets and base URLs

Target	Base URL used by Flutter	WebSocket URL	Backend requirement
Android emulator	http://10.0.2.2:8000	ws://10.0.2.2:8000/ws/live/	10.0.2.2 in ALLOWED_HOSTS
iOS simulator	http://127.0.0.1:8000	ws://127.0.0.1:8000/ws/live/	Already allowed
Physical phone (same Wi-Fi)	http://<computer IP>:8000	ws://<computer IP>:8000/ws/live/	Computer IP in ALLOWED_HOSTS; runserver 0.0.0.0:8000
Flutter web (Chrome)	http://127.0.0.1:8000	ws://127.0.0.1:8000/ws/live/	Run Flutter web on port 3000 (already in CORS_ALLOWED_ORIGINS)

13.3 Backend requirements for Flutter

ID	Requirement	Status
BE-01	ALLOWED_HOSTS includes localhost, 127.0.0.1, 10.0.2.2 and the computer's Wi-Fi IP during testing. With ALLOWED_HOSTS = [] Django rejects the emulator and phones with 400 Bad Request.	Done (Phase 10)
BE-02	CORS allows the Flutter web origin (http://localhost:3000 and http://127.0.0.1:3000). Native apps are not affected by CORS.	Done (no change needed)
BE-03	Login and /api/accounts/me/ return role so the app can open the right screens.	Done (confirmed)
BE-04	Every response follows Section 10, including validation errors per field.	Done (confirmed)
BE-05	Long operations return 202 with a task_id that can be polled.	Done (confirmed)
BE-06	The Celery app is imported in stock_risk_engine/__init__.py so queued jobs use the project's Celery settings.	Done (Phase 10)
BE-07 (optional)	Read-only admin APIs for dataset uploads, training runs and model versions.	Not added; the Flutter admin links to the web pages
BE-08	Live risk monitoring WebSocket as specified in Section 12.	Phase 10B

13.4 App architecture

Concern	Choice
HTTP	dio with an interceptor that adds the Bearer token and handles 401 by refreshing once
Live connection	web_socket_channel in a LiveSocket service: connect, ping every 30 seconds, reconnect with back-off, expose an event stream and the connection state
Token storage	flutter_secure_storage (never plain shared preferences)
State	provider for the logged-in user, role and live connection state (simple and beginner-friendly)
Navigation	go_router with redirects by login state and role
Charts	fl_chart for price trends
Formatting	intl for money, percentages and dates
Configuration	--dart-define=API_BASE_URL=... per target; the WebSocket URL is derived from it (http → ws)
Android testing	android:usesCleartextTraffic="true" in debug builds so plain HTTP and WS work locally

13.5 Authentication and session requirements

ID	Requirement
FL-01	Splash screen reads stored tokens, calls /api/accounts/me/ and opens the home screen for the user's role, or Login.
FL-02	Login accepts username or email; on success both tokens are stored securely.
FL-03	Register creates an investor and then shows Login (or logs in automatically).
FL-04	On 401 the app refreshes the access token once and retries; if refresh fails it clears tokens and opens Login.
FL-05	Logout calls /api/accounts/logout/ with the refresh token, closes the live connection and clears storage.
FL-06	After a password change the new token pair from the response replaces the old one and the live connection reconnects.
FL-07	Screens and buttons a role cannot use are not shown.
FL-08	The live connection opens after login or splash and pings every 30 seconds.
FL-09	On close code 4401 the app refreshes the token once and reconnects; on other drops it reconnects with back-off and subscribes again.
FL-10	Screens subscribe to the tickers they show and unsubscribe when closed.
FL-11	After a recompute the app waits for prediction.updated with the same task_id; after 30 seconds without it, it polls /api/analytics/tasks/{task_id}/.

13.6 Screen inventory — common

Screen	API	Content
Splash	GET accounts/me/; WS connect	Route by role; open the live connection
Login	POST accounts/login/	Username or email, password, link to Register
Register	POST accounts/register/	Username, email, password, confirm password
Profile	GET / PATCH accounts/me/	Username, email, role badge, assigned analyst, logout
Change password	POST accounts/change-password/	Old, new and confirm password

13.7 Screen inventory — investor

Screen	API	Content
Home	GET dashboard/investor/; WS price.update, risk.changed	Total value (updates live), gain/loss, portfolio count, risk distribution, portfolio cards, risk alert banner
Portfolios	GET portfolios/	List with holdings count and value; pull-to-refresh; add button
Portfolio form	POST portfolios/, PATCH portfolios/{id}/	Name, description
Portfolio detail	GET portfolios/{id}/summary/, GET analytics/portfolio/{id}/risk-summary/; WS subscribe to its tickers	Value, cost, gain/loss, risk and direction distribution, holdings list with live price and day change
Holding form	POST portfolios/{id}/holdings/, PATCH holdings/{id}/	Ticker (create only), quantity, buy price, buy date
Holding detail	GET holdings/{id}/, analytics risk/ and forecast/, dashboard/holdings/{id}/trend/; WS subscribe	Live price chip (Live / Daily close), value, gain/loss, risk badge and confidence, direction and confidence, model versions, price chart with buy price line
Stock lookup	GET market/prices/{ticker}/ and history/; WS subscribe	Latest price, recent history and a Follow button

13.8 Screen inventory — analyst

Screen	API	Content
Home	GET dashboard/analyst/; WS risk.changed	Client count, total value, high-risk holdings list, live alert list for assigned clients
Clients' portfolios	GET portfolios/	Only assigned investors' portfolios, read-only
Portfolio and holding detail	Same as investor	No add, edit or delete controls
Recompute prediction	POST analytics/holdings/{id}/recompute/; WS prediction.updated (fallback GET analytics/tasks/{task_id}/)	Button on holding detail; shows progress until the event arrives, then updates
Refresh prices	POST market/refresh/{ticker}/	Button on holding detail; shows a message when queued; handles 429

13.9 Screen inventory — admin

Screen	API	Content
Home	GET dashboard/admin/; WS risk.changed	Users by role, portfolio and holding counts, tickers tracked, latest price date, models in use, live alert feed for all holdings
Users	GET accounts/users/	Paginated list with role badges and assigned analyst
Edit user	PATCH accounts/users/{id}/	Role picker; analyst picker for investors; own role locked
All portfolios	GET portfolios/	Every portfolio with owner
Data and training	Opens /datasets/ and /ml/ in the browser	Upload and training stay in the Django web pages

13.10 Page design requirements

ID	Requirement
UI-01	Material 3 with one seed colour and light and dark themes.
UI-02	Designed for phone width first; layouts also work on tablets and in Chrome.
UI-03	Bottom navigation per role — investor: Home, Portfolios, Stocks, Profile; analyst: Home, Clients, Profile; admin: Home, Users, Portfolios, Profile.
UI-04	Same colour meaning as the web pages — risk: low green, medium amber, high red, not rated grey; direction: up green, down red. Labels are always shown as text too.
UI-05	Money uses two decimals and a currency symbol; gains show + in green and losses − in red.
UI-06	Every screen has loading, empty, error-with-retry and success states.
UI-07	Lists support pull-to-refresh and load more pages using the next link.
UI-08	Forms show the API's field errors under each field and disable the submit button while sending.
UI-09	Delete actions ask for confirmation.
UI-10	Predictions show confidence, model version and last prediction time, with a note that they are not financial advice.
UI-11	Values not yet available (no price, no prediction) show a clear placeholder such as 'Not rated yet'.
UI-12	A small connection indicator (connected, reconnecting, offline) in the app bar; when offline, screens keep the last data and show 'Live updates paused'.
UI-13	Live prices briefly highlight when they change and always show 'Live' or 'Daily close'.
UI-14	Risk alerts show the old and new label as text, for example 'AAPL: medium → high', and open the holding when tapped.

13.11 Acceptance criteria for the Flutter test

- The full journey works in the app: register an investor, admin promotes another user to analyst and assigns them, investor creates a portfolio and adds a holding, risk and forecast appear after predictions run, the analyst sees the portfolio read-only and can recompute, the token expires and refreshes without the user noticing, logout works.

- With the investor's holding open, an analyst recompute updates the investor's screen without a refresh; a forced label change shows an alert to the investor, their analyst and an admin only.
- Turning Wi-Fi off and on reconnects and resubscribes automatically; after 15 minutes the socket closes with 4401 and reconnects with a refreshed token.
- An investor never sees another investor's data; an analyst cannot edit; admin screens are hidden from other roles.
- The app works on the Android emulator, a physical phone and Chrome.

14. Celery and Redis Task Design

14.1 Redis databases

DB	Use
0	Celery broker
1	Celery results
2	Django cache, throttle counters and live quotes
3	Refresh-token blacklist
4	Channel layer (WebSocket messages between workers and connections)

Settings refuse to start if the five URLs are not different, so clearing the cache can never delete queued tasks, logged-out tokens or WebSocket messages.

14.2 Scheduled and on-demand tasks

Task	Trigger	Purpose	Live event
core.tasks.ping	Every 15 minutes	Worker health check	—
monitoring.tasks.poll_live_prices	Every 2 minutes, Mon–Fri 13:00–21:58 UTC	Live quotes for held tickers	price.update (live)
market_data.tasks.fetch_daily_prices	Mon–Fri 22:00 UTC; Fetch now button	Download missing daily prices	price.update (daily) for tickers with new rows
market_data.tasks.refresh_single_ticker	Refresh API; new holding	One ticker, retries with back-off	price.update (daily) when rows are added
market_data.tasks.import_dataset_upload	Datasets page	Import uploaded files with progress	—
analytics.tasks.recompute_holding_risk	Mon–Fri 23:00 UTC	Risk predictions for all holdings	risk.changed for labels that moved
analytics.tasks.recompute_holding_forecast	Mon–Fri 23:15 UTC	Direction predictions for all holdings	—
analytics.tasks.recompute_single_holding	Recompute API	Both predictions for one holding	prediction.updated; risk.changed if the label moved
ml_training.tasks.run_training	ML Training page	Train the chosen models	risk.changed when predictions are updated
ml_training.tasks.retrain_models	Sunday 02:00 UTC	Weekly retraining recorded as a TrainingRun	risk.changed when predictions are updated

14.3 Caching

Data	TTL	Cleared when
Portfolio summary, portfolio risk summary	300 seconds	Portfolio or holding changes, new prices for a held ticker, new predictions
Investor, analyst and admin dashboards	300 seconds	The same events, plus registrations, role and assignment changes, training runs
Live quote per ticker	24 hours	Replaced by every new live or daily price.update

15. Machine Learning Pipeline

15.1 Code location

File (ml_training/)	Contents
algorithms.py	RandomForestClassifier, LogisticRegression, HistGradientBoostingClassifier and their settings
feature_engineering.py	The eight features
labels.py	Risk and direction labels
dataset.py	Loading prices and the time-based split
train_risk.py, train_forecast.py	Training, evaluation and baseline comparison
registry.py	Saving, versioning and loading models
services.py, tasks.py, views.py	Training runs from the web page and the weekly schedule

15.2 Features

Calculated per ticker using only past daily prices, so adding new prices never changes earlier values: daily_return, volatility_30, volatility_90, max_drawdown_90, price_to_ma_20, price_to_ma_50, rsi_14, volume_trend. Live intraday quotes are not used as features.

15.3 Model 1 — Risk classification

- Label: standard deviation of daily returns over the next 30 trading days, split into low / medium / high at the training period's 33rd and 66th percentiles.
- Model: RandomForestClassifier. Baseline: LogisticRegression, compared on macro F1.
- Output: risk_label and risk_score (probability of the predicted class). A change of risk_label sends risk.changed.

15.4 Model 2 — Direction forecast

- Label: up if the next trading day's close is above today's close, otherwise down.
- Model: HistGradientBoostingClassifier. Baselines: always up, and same direction as today.
- Output: predicted_direction and prediction_confidence.

15.5 Training method and evaluation

Split	Dates
Train	Up to 31 December 2022
Validation	2023
Test	2024 onwards

- The risk split leaves a 45-day gap before each period so 30-day labels never overlap the next period.
- Metrics: accuracy, macro F1, per-class precision/recall/F1 and confusion matrix for risk; accuracy, F1 and ROC-AUC for direction.
- Next-day direction accuracy of about 50–55% is normal and is reported honestly against the baselines.

15.6 Versioning

Each run saves ml_models/<risk|forecast>/<version>.joblib with a JSON metadata file (features, date windows, label thresholds, parameters, metrics, baseline comparison). Versions look like risk-rf-2026.11.07-01. Predictions always use the newest version, and each holding stores the version that produced it; risk.changed and prediction.updated events carry those versions.

16. Dataset Source and Ingestion

The historical data comes from the Kaggle dataset S&P 500 Stocks, Daily Historical Data (10 Years): one CSV per ticker with Date, Open, High, Low, Close and Volume. Each row maps directly onto a price_history row, and ten years of daily data covers the training (to 2022), validation (2023) and test (2024 onwards) periods with enough rows for 90-day features and 30-day labels.

- Uploaded through the Datasets page (Section 11.1); no command line is needed.
- Invalid rows (bad dates, non-positive prices, high below low) are skipped and counted.
- Daily updates after the upload come from Yahoo Finance; live quotes during market hours also come from Yahoo Finance but are only cached (Section 12.7).

17. Environment Configuration

Variable	Purpose
DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT	PostgreSQL connection
SECRET_KEY, JWT_SECRET_KEY, JWT_REFRESH_SECRET_KEY	Three different secrets
ACCESS_TOKEN_LIFETIME_MINUTES, REFRESH_TOKEN_LIFETIME_DAYS	15 and 7
CELERY_BROKER_URL, CELERY_RESULT_BACKEND	Redis databases 0 and 1
REDIS_URL, TOKEN_BLACKLIST_REDIS_URL	Redis databases 2 and 3
CHANNEL_LAYER_REDIS_URL	Redis database 4 for WebSocket messages (new in version 3.0)
CACHE_TTL_SECONDS	Summary cache lifetime (300)
LOGIN_THROTTLE_RATE, REGISTER_THROTTLE_RATE, MARKET_REFRESH_THROTTLE_RATE	5/minute, 10/hour, 10/hour
MARKET_SEED_TICKERS	Tickers always included in daily price updates

Fixed settings in settings.py: daphne first in INSTALLED_APPS, ASGI_APPLICATION, CHANNEL_LAYERS (Redis channel layer with socket_timeout 10), ML_MODELS_DIR (ml_models/), MEDIA_ROOT (media/), DATA_UPLOAD_MAX_NUMBER_FILES = 1000 (so a full S&P 500 folder can be uploaded), TIME_ZONE = UTC, and ALLOWED_HOSTS / CORS_ALLOWED_ORIGINS as in Section 13.3.

18. Project Directory Structure

```
stock_risk_engine/
├── accounts/          user model, JWT, auth APIs and pages, role management
├── analytics/         prediction services, tasks and APIs
├── core/              logger, decorators, responses, permissions, cache, throttles, fields
├── dashboards/        dashboard APIs and pages, portfolio and holding pages
├── market_data/       price history, dataset uploads, Yahoo Finance, price APIs
├── ml_training/       algorithms, features, labels, training, registry, training pages
├── monitoring/        WebSocket consumer, routing, form, live events, live price task
├── portfolios/        portfolio and holding models, APIs, summary
├── stock_risk_engine/ settings.py, urls.py, celery.py, asgi.py (HTTP + WebSocket)
├── templates/         base, accounts, core, dashboards, market_data, ml_training
├── script/js/         page scripts and price chart
├── ml_models/         saved models (git-ignored)
├── media/             uploads while they are imported (git-ignored)
├── logs/              app.log (git-ignored)
├── .env.example
├── requirements.txt
└── stock_risk_engine.postman_collection.json

stock_risk_engine_flutter/ (separate project, Section 13)
└── lib/ config/ core/ (api_client, token_storage, live_socket) models/ services/
        screens/{common,investor,analyst,admin}/ widgets/
```

19. Security and Non-Functional Requirements

Area	Requirement
Data isolation	Querysets are scoped by role; outside records return 404; live holding events reach only the owner, the assigned analyst and admins
Tokens	Short-lived access tokens, separate secrets, refresh blacklist, invalidation on password change; Flutter stores tokens in secure storage
WebSocket	Access token required; closes with 4401 on expiry or account change; at most 20 followed tickers; messages validated by a form; token never logged by the server
Rate limiting	Login, registration and price refresh limits with retry_after_seconds
Uploads	Admin only; .csv and .zip only; file size limits; files deleted after import; zip entries read without extracting to arbitrary paths
ML outputs	Written only by background tasks; never accepted from clients
Performance	Joined and annotated queries, Redis caching of summaries and dashboards, background processing of slow work, one live connection per app instead of polling
Reliability	A failed event publish is logged and never fails the task that produced it; REST remains the source of truth
Observability	Rotating logs/app.log with logins, denials, cache activity, uploads, training runs, task results, live connections and alerts, and timings
Environment	Local development with DEBUG = True; production settings (HTTPS / WSS, environment-driven hosts) only when deploying (Section 20)

20. Delivery Roadmap and Future Scope

Phase	Scope	Status
0–9	Foundation, accounts, core, market data, portfolios, ML, analytics, dashboards, hardening, testing	Complete
9B	GUI dataset upload and training, ml_training app, config merged into core	Complete
10	Flutter readiness on the backend: ALLOWED_HOSTS for emulator and phone, CORS check, Celery import, API contract confirmed (Section 13.3)	Complete
10B	Live risk monitoring WebSocket API: Channels, Daphne, channel layer, monitoring app, live price task, risk and recompute events (Section 12)	Next
11	Flutter test client (Section 13): 11A setup and login; 11B investor; 11C analyst; 11D admin; 11E live layer; 11F polish and device testing	Planned
12	Docker deployment: Daphne web (HTTP and WebSocket), Celery worker and Beat, PostgreSQL, Redis; HTTPS and WSS through a reverse proxy	Planned
13	Future scope (below)	Planned

- Push notifications (Firebase Cloud Messaging) when a risk label changes while the app is closed; the WebSocket covers the app while it is open.
- User alert preferences: which holdings or which risk levels send alerts.
- A paid streaming price provider for second-by-second quotes instead of 2-minute polling.
- Sector analytics and diversification score.
- Rebalancing suggestions based on composition and risk.

- LSTM forecasting as a comparison against Gradient Boosting.
- Choosing an older model version as the active one.

Appendix A — Sample Flutter Test Flow

Step	Who	Action	API / page
1	Admin	Upload the Kaggle zip and wait for completion	Web: /datasets/
2	Admin	Train both models with 'update predictions'	Web: /ml/
3	Investor (Flutter)	Register, then log in; the live connection opens	POST register/, POST login/, WS connect
4	Analyst (Flutter)	Register (becomes an investor)	POST register/
5	Admin (Flutter)	Promote the analyst and assign them to the investor	PATCH users/{id}/ ×2
6	Investor (Flutter)	Create a portfolio and add AAPL	POST portfolios/, POST portfolios/{id}/holdings/
7	Investor (Flutter)	Open the AAPL holding; the live price chip appears	GET holding, risk, forecast, trend; WS subscribe
8	Analyst (Flutter)	Open the client's holding and tap Recompute	POST recompute/; WS prediction.updated
9	Investor (Flutter)	The holding updates without a refresh; a risk alert appears if the label moved	WS prediction.updated, risk.changed
10	Investor (Flutter)	Keep using the app past 15 minutes; token and live connection refresh silently	POST token/refresh/; WS reconnect
11	Investor (Flutter)	Log out; the old refresh token is rejected and the live connection closes	POST logout/

Appendix B — Machine Learning Disclaimer

Risk labels and direction forecasts are statistical estimates based on historical daily prices. They are shown with their confidence and model version and are not financial advice. Next-day direction accuracy near 50% is expected for daily stock data. Live price updates do not change a prediction until the next daily run.